

Mathematical Modeling of 2D Incompressible Fluid Flow

Ş. K. Alkır

1 Governing equations

Let $\Omega = [-1, 1]^2 \subset \mathbb{R}^2$ be a unit square with Cartesian coordinates $\mathbf{x} = (x, y)$, and let $t \geq 0$ denote time. The unknown fields are

$$\rho(\mathbf{x}, t) \in \mathbb{R}_{\geq 0}, \quad \mathbf{u}(\mathbf{x}, t) = (u, v)^\top \in \mathbb{R}^2, \quad p(\mathbf{x}, t) \in \mathbb{R}.$$

They satisfy the incompressible Navier–Stokes system with a passive-scalar density:

$$\partial_t \rho + \mathbf{u} \cdot \nabla \rho = D_\rho \Delta \rho + S_\rho, \tag{1a}$$

$$\partial_t \mathbf{u} + (\mathbf{u} \cdot \nabla) \mathbf{u} = -\nabla p + \nu \Delta \mathbf{u} + \mathbf{f}, \tag{1b}$$

$$\nabla \cdot \mathbf{u} = 0. \tag{1c}$$

1.1 Physical interpretation and modelling rationale

(i) Passive–scalar transport. Treat (1a) as a *control–volume budget*. The left–hand side is the *material derivative* of the concentration: it measures the instantaneous inventory change of a moving fluid parcel. On the right, $D_\rho \nabla^2 \rho$ is the divergence of the diffusive flux (Fick’s law), spreading the scalar down its own gradient, while S_ρ is an external source representing user injections. In short, **material change = diffusive import + prescribed input**—a microscopic statement of the first law for a passive tracer.

(ii) Momentum balance (Navier–Stokes). The equation (1b) is Newton’s second law written *per unit mass*. The material acceleration on the left is the “expense column” in our ledger; it quantifies how rapidly a fluid element’s velocity state is being updated. The right-hand “revenue column” itemises the forces that can pay that expense: $-\nabla p$ is the reversible work done by pressure, $\nu \nabla^2 \mathbf{u}$ is the irreversible viscous diffusion of momentum, and $\mathbf{f}$ is an externally imposed specific force (our programmable wind). The equality therefore asserts a pointwise force balance: **material acceleration = pressure drive + viscous drag + external forcing**.

(iii) Incompressibility constraint. Finally, (1c) is the local volume-conservation ledger. The divergence operator tallies “flow-in minus flow-out” for an infinitesimal cube; setting that tally to zero demands that no fluid mass is created or destroyed and that density remains constant along trajectories. This requirement motivates the projection step used later (Section 6) and underlies the ubiquitous “pressure–Poisson” formulation in fluid solvers. Thus, the statement is a strict bookkeeping rule: **incoming volumetric flux – outgoing volumetric flux = 0**, everywhere and always, anchoring the solver to the physics of incompressible flow.

Why projection? Equation (1c) imposes a *pointwise* divergence-free constraint. A direct time-stepping of (1b) would not preserve $\nabla \cdot \mathbf{u} = 0$ at the discrete level. Chorin’s projection method resolves this by splitting the update into a provisional step followed by an orthogonal projection onto the space of solenoidal vector fields (Section 6).

2 Spatial discretisation

2.1 Centered grid & indexing

The domain is tessellated by a uniform Cartesian grid with N interior cells per direction; in code $N = 200$. For simplicity we store *all* variables at cell centres (**stable fluids** layout) and add a *one-cell halo* to encode boundary conditions. The linear index

$$\text{IX}(i, j) = i + (N + 2)j, \quad 0 \leq i, j \leq N + 1,$$

maps the 2-D lattice to the 1-D `Float32Array` buffers (`u`, `v`, `...`). With $(\Delta x, \Delta y) = (h, h)$ and $h = 1/N$, grid points are located at $(x_i, y_j) = ((i - \frac{1}{2})h, (j - \frac{1}{2})h)$ for $1 \leq i, j \leq N$.

3 Temporal discretisation

A fixed Eulerian time step $\Delta t = \text{dt} = 0.01$ is used. The algorithm is operator-split into the following sub-steps (cf. Algorithm 1):

1. **Velocity forcing** by the wind field $\mathbf{f}$ (explicit Euler).
2. **Diffusion** of $\mathbf{u}$ via an *implicit* (unconditionally stable) solve §4.
3. **Projection** to enforce $\nabla \cdot \mathbf{u} = 0$ (§6).
4. **Advection** of $\mathbf{u}$ using semi-Lagrangian backtracing (§5).
5. **Second projection** (Strang-type splitting).
6. **Transport** of ρ : diffusion then advection.
7. **Artificial damping** (velocity, density, boundary).

Each step is mathematically well defined and conservative with respect to the quantities it advances; the only explicit dissipations are the physical ν, D_ρ and the user-adjustable damping factors $\gamma_v < 1, \gamma_\rho < 1$.

4 Diffusion step

4.1 Implicit time integration and unconditional stability

For a generic scalar field $\phi \in \{u, v, \rho\}$, representing either a velocity component or the passive density, the physical model reads

$$\partial_t \phi = \alpha \Delta \phi, \quad \alpha > 0. \quad (2)$$

Let $t^n = n\Delta t$ and denote by $\phi_{i,j}^n \approx \phi(x_i, y_j, t^n)$ the grid solution at $(i, j) \in \{1, \dots, N\}^2$. Backward Euler in time combined with the five-point discrete Laplacian gives

$$\phi_{i,j}^{n+1} - \phi_{i,j}^n = a \left(\phi_{i-1,j}^{n+1} + \phi_{i+1,j}^{n+1} + \phi_{i,j-1}^{n+1} + \phi_{i,j+1}^{n+1} - 4\phi_{i,j}^{n+1} \right), \quad a := \frac{\alpha \Delta t}{h^2}. \quad (3)$$

Symmetric positive definiteness. Rewriting (3) in vector form $(I - aL)\phi^{n+1} = \phi^n$, the matrix $A := I - aL$ is an M-matrix: symmetric, strictly diagonally dominant, and therefore positive definite for all $a > 0$. This guarantees uniqueness of ϕ^{n+1} and makes the Jacobi (point) iteration converge geometrically. Moreover, the scheme is *unconditionally L^2 -stable*. Indeed, multiplying (3) by $\phi_{i,j}^{n+1}$ and summing over all grid indices yields the discrete energy decay

$$\|\phi^{n+1}\|_2^2 - \|\phi^n\|_2^2 = -a \|\nabla_h \phi^{n+1}\|_2^2 \leq 0,$$

so no CFL restriction on Δt is required.

Boundary operators. The helper `setBoundary` enforces:

$$\begin{aligned}\phi_{0,j} &= \pm \phi_{1,j}, & \phi_{N+1,j} &= \pm \phi_{N,j}, \\ \phi_{i,0} &= \pm \phi_{i,1}, & \phi_{i,N+1} &= \pm \phi_{i,N},\end{aligned}\quad 1 \leq i, j \leq N,$$

with the sign choice $+$ (Neumann) for scalar fields and the sign $-$ (Dirichlet) for the normal velocity components; this choice preserves the discrete energy estimate used in the stability argument.

4.2 Consistency and global error

The truncation error of (3) is $\mathcal{O}(\Delta t + h^2)$, so the scheme is first-order accurate in time and second-order in space. Combined with the L^2 -stability above, the Lax–Richtmyer theorem yields global error $\mathcal{O}(\Delta t + h^2)$ for smooth solutions.

5 Advection step

5.1 Geometric interpretation and continuous formulation

Let $\mathbf{w} : \Omega \times [0, T] \rightarrow \mathbb{R}^d$ denote a scalar or vector field being transported by a time-dependent velocity $\mathbf{u}(\mathbf{x}, t)$. The advection equation

$$\partial_t \mathbf{w} + \mathbf{u} \cdot \nabla \mathbf{w} = 0 \quad \text{with} \quad \mathbf{w}(\mathbf{x}, t_0) = \mathbf{w}^0(\mathbf{x}) \quad (4)$$

This means that as each fluid parcel moves through space, it carries its initial field value with it unchanged. To describe this transport formally, we introduce the *flow map* $\mathbf{X}(t_0; t, \mathbf{x})$, which gives the position at time t_0 of the fluid particle that arrives at location $\mathbf{x}$ at time t .

Then the solution to (4) at time $t^{n+1} = t^n + \Delta t$ is:

$$\mathbf{w}^{n+1}(\mathbf{x}) = \mathbf{w}^n(\mathbf{X}(t^n; t^{n+1}, \mathbf{x})), \quad (5)$$

This is conceptually natural: to find the field value at point $\mathbf{x}$ and time t^{n+1} , trace the characteristic backwards to where the fluid parcel came from, and recover its earlier value there.

5.2 Semi-Lagrangian discretisation

Computing the departure point $\mathbf{X}(t^n; t^{n+1}, \mathbf{x})$ exactly would mean following the particle's path through the velocity field for an entire time step—a costly operation in real time. Instead, we approximate the backward trajectory by a single explicit Euler step:

$$\tilde{\mathbf{x}} = \mathbf{x} - \Delta t \mathbf{u}^n(\mathbf{x}).$$

Because $\tilde{\mathbf{x}}$ is almost never located *exactly* on a grid node, we must estimate the previous field value $\mathbf{w}^n$ there. Define the interpolation operator $\mathcal{I} : \{\mathbf{w}_{i,j}^n\} \rightarrow C^0(\Omega)$ by

$$\mathcal{I}[\mathbf{w}^n](x, y) := \sum_{m=0}^1 \sum_{n=0}^1 \lambda_{mn}(x, y) \mathbf{w}_{i+m, j+n}^n,$$

where (i, j) is the lower-left grid node of the cell that contains (x, y) and the bilinear shape functions are

$$\lambda_{00} = (1 - \xi)(1 - \eta), \quad \lambda_{10} = \xi(1 - \eta), \quad \lambda_{01} = (1 - \xi)\eta, \quad \lambda_{11} = \xi\eta,$$

with $\xi = \frac{x - x_i}{h} \in [0, 1]$, $\eta = \frac{y - y_j}{h} \in [0, 1]$. Each $\lambda_{mn} \geq 0$ and $\sum_{m,n} \lambda_{mn} = 1$, so the operator is *contractive* in every L^p norm.

The semi-Lagrangian update therefore reads

$$\mathbf{w}^{n+1}(\mathbf{x}) = \mathcal{I}[\mathbf{w}^n](\tilde{\mathbf{x}}), \quad (6)$$

i.e. “sample the old field at the estimated upstream location.” This construction mimics a *Lagrangian* perspective—track what a fluid particle carries—while keeping the data fixed on an Eulerian grid.

The key advantage is unconditional stability: regardless of Δt , the method cannot introduce new extrema, since it simply samples a convex combination of previous values.

Stability. Because update (6) merely interpolates existing data, it is contractive in every L^p norm, $\|\mathbf{w}^{n+1}\|_p \leq \|\mathbf{w}^n\|_p$ for $1 \leq p \leq \infty$; hence no Courant restriction and no spurious extrema arise. This unconditional stability—at first-order accuracy in time and second-order in space—is the key trade-off that makes the method ideal for real-time graphics.

6 Pressure projection

At each time step, the provisional velocity field $\mathbf{u}^*$ produced by forcing, diffusion, or advection may violate the incompressibility constraint $\nabla \cdot \mathbf{u} = 0$. To restore divergence-freeness, we perform a *projection* onto the subspace of solenoidal fields.

Helmholtz decomposition and pressure correction

Let $\mathbf{v} : \Omega_h \rightarrow \mathbb{R}^2$ be a discrete vector field defined on a regular Cartesian grid Ω_h with uniform spacing $h > 0$. A fundamental result in vector calculus, known as the *Helmholtz decomposition*, states that any sufficiently regular vector field can be uniquely decomposed as the sum of a divergence-free component and a gradient field. On the discrete grid, this takes the form

$$\mathbf{v} = \mathbf{v}_{\text{div}} + \nabla_h \pi, \quad (7)$$

where:

- $\mathbf{v}_{\text{div}}$ is a vector field satisfying $\nabla_h^* \mathbf{v}_{\text{div}} = 0$, i.e., it is discretely divergence-free,
- $\nabla_h \pi$ is the discrete gradient of a scalar field $\pi : \Omega_h \rightarrow \mathbb{R}$,
- ∇_h^* denotes the discrete divergence operator, defined as the negative adjoint of the gradient with respect to the grid inner product.

This decomposition is orthogonal in the discrete L^2 inner product and is unique up to an additive constant in π .

On the regular grid with spacing h , the discrete gradient operator ∇_h maps scalar grid functions $\pi_{i,j}$ to vector fields via central differences:

$$(\nabla_h \pi)_{i,j} = \left(\frac{\pi_{i+1,j} - \pi_{i-1,j}}{2h}, \frac{\pi_{i,j+1} - \pi_{i,j-1}}{2h} \right).$$

The discrete divergence operator ∇_h^* acts on a vector field $\mathbf{v}_{i,j} = (u_{i,j}, v_{i,j})$ as:

$$(\nabla_h^* \mathbf{v})_{i,j} = \frac{u_{i+1,j} - u_{i-1,j}}{2h} + \frac{v_{i,j+1} - v_{i,j-1}}{2h}.$$

These satisfy the identity $\langle \mathbf{v}, \nabla_h \pi \rangle_h = -\langle \nabla_h^* \mathbf{v}, \pi \rangle_h$ under the grid L^2 inner product, confirming that ∇_h^* is indeed the negative adjoint of ∇_h .

In the context of incompressible fluid simulation, we apply this decomposition to the provisional velocity field $\mathbf{u}^*$, which results from the combination of advection, external forcing, and viscosity but does not generally satisfy the incompressibility condition $\nabla \cdot \mathbf{u} = 0$.

To recover a physically admissible velocity field $\mathbf{u}^{n+1}$ that is divergence-free, we define:

$$\mathbf{u}^{n+1} := \mathbf{u}^* - \nabla_h \pi,$$

where π is chosen such that the divergence of $\mathbf{u}^{n+1}$ vanishes:

$$\nabla_h^* \mathbf{u}^{n+1} = \nabla_h^* (\mathbf{u}^* - \nabla_h \pi) = \nabla_h^* \mathbf{u}^* - \Delta_h \pi = 0.$$

This leads to the discrete Poisson equation for the pressure correction:

$$\Delta_h \pi = \nabla_h^* \mathbf{u}^*, \tag{8}$$

where $\Delta_h := \nabla_h^* \nabla_h$. In practice, the Poisson equation (8) is solved numerically using the same iterative Jacobi (point) method as in the diffusion step; see Section 4.

We impose homogeneous Neumann boundary conditions for π , corresponding to the physical condition of no normal flow at the domain boundaries:

$$\partial_n \pi = 0 \quad \text{on } \partial\Omega_h.$$

Since π is determined only up to an additive constant, the solution to (8) is not unique. However, the velocity field $\mathbf{u}^{n+1}$ is fully determined, because the gradient of a constant vanishes.

In summary, the pressure correction π serves to remove the non-solenoidal component of the provisional velocity field $\mathbf{u}^*$, producing an updated velocity $\mathbf{u}^{n+1}$ that satisfies the incompressibility condition exactly at the discrete level.

This projection step is applied both before and after the advection stage as part of a Strang-type operator splitting; see Section 9 for the full time-stepping sequence.

Stability and energy estimate

By orthogonality of the decomposition, the projection does not increase kinetic energy:

$$\|\mathbf{u}^{n+1}\|_2 = \|\mathbf{u}^*\|_2 - \|\nabla_h \pi\|_2,$$

ensuring that no artificial energy is introduced when enforcing incompressibility. This mirrors the continuous L^2 -energy estimate and preserves the dissipative nature of the Navier–Stokes evolution.

The discrete L^2 norm is defined as

$$\|\mathbf{u}\|_2 := h^2 \sum_{i,j} (u_{i,j}^2 + v_{i,j}^2),$$

with the corresponding inner product $\langle \mathbf{u}, \mathbf{v} \rangle := h^2 \sum_{i,j} (u_{i,j} v_{i,j} + v_{i,j} w_{i,j})$.

7 Wind forcing

The external force field $\mathbf{f}(x, y)$, referred to as the wind field, is defined pointwise on the domain according to one of the following user-selectable modes:

$$\mathbf{f}(x, y) = \begin{cases} f_0 \begin{pmatrix} \cos \theta \\ \sin \theta \end{pmatrix}, & \text{Uniform (constant direction),} \\ f_0 \begin{pmatrix} -\hat{y}(x, y) \\ \hat{x}(x, y) \end{pmatrix}, & \text{Vortex (solid-body rotation),} \\ f_0 \begin{pmatrix} \sin(\pi x) \\ \cos(\pi y) \end{pmatrix}, & \text{Sinusoidal,} \\ f_0 \begin{pmatrix} f(x, y) \\ g(x, y) \end{pmatrix}, & \text{Custom (user-defined),} \end{cases}$$

where:

- $f_0 \in [0, 20]$ is the user-defined magnitude of the wind,
- $\theta \in [0, 2\pi)$ specifies the angle of uniform flow,
- $(\hat{x}(x, y), \hat{y}(x, y)) := \frac{(x-x_c, y-y_c)}{\sqrt{(x-x_c)^2 + (y-y_c)^2}}$ is the unit radial vector from the domain center (x_c, y_c) (used in the vortex mode),
- $f(x, y)$ and $g(x, y)$ are arbitrary scalar functions supplied by the user in the Custom mode.

Note that all functions are defined over the rescaled domain $[-1, 1]^2$, where the origin corresponds to the center of the canvas.

8 Density injection and damping

User interaction is implemented as an external source term in the density equation (1a). Specifically, a mouse drag triggers the addition of a source term

$$S_\rho(x, y, t) = \rho_0 \mathbf{1}_{B_r(x_0, y_0)}(x, y),$$

where ρ_0 is the prescribed injection strength, and $B_r(x_0, y_0)$ is a ball of radius r centered at the injection location (x_0, y_0) . The indicator function $\mathbf{1}_{B_r}$ ensures that the density is injected only within this local region. In addition to modifying ρ , a small velocity impulse is applied to $\mathbf{u}$ in the direction of the drag, simulating physical stirring.

To prevent unbounded accumulation of density and numerical artefacts during long-time integration, artificial damping is applied after each time step:

$$\mathbf{u} \leftarrow \gamma_v \mathbf{u}, \quad \rho \leftarrow \gamma_\rho \rho, \quad \text{with } \gamma_v, \gamma_\rho \lesssim 1.$$

These coefficients serve as phenomenological models for subgrid-scale dissipation and help stabilize the simulation while preserving large-scale dynamics. Both γ_v and γ_ρ are user-configurable parameters.

9 Algorithmic summary

Algorithm 1 Advance one time step $t^n \rightarrow t^{n+1}$

```

1: procedure STEP
2:   if mouse is dragging then
3:     Inject:  $\rho += S_\rho, u += \Delta u, v += \Delta v$            ▷ density source and stirring impulse
4:   end if
5:   Force:  $u[k] += \Delta t \text{ windU}[k], v[k] += \Delta t \text{ windV}[k]$            ▷ apply wind field
6:   Diffuse: DIFFUSE(1,  $u_0, u, \nu$ ); DIFFUSE(2,  $v_0, v, \nu$ )           ▷ implicit viscous solve
7:   Project: PROJECT( $u_0, v_0, p, \text{div}$ )           ▷ enforce  $\nabla \cdot \mathbf{u} = 0$ 
8:   Advect: ADVECT(1,  $u, u_0, u_0, v_0$ ); ADVECT(2,  $v, v_0, u_0, v_0$ )           ▷ semi-Lagrangian
   transport
9:   Project: PROJECT( $u, v, p, \text{div}$ )           ▷ remove post-advection divergence
10:  Density steps: DIFFUSE(0,  $\rho_0, \rho, D_\rho$ ); ADVECT(0,  $\rho, \rho_0, u, v$ )           ▷ diffuse then advect
   scalar
11:  Damping:  $u \leftarrow \gamma_u u, v \leftarrow \gamma_v v, \rho \leftarrow \gamma_\rho \rho$            ▷ global decay
12:  Edge damping: halve  $\rho$  on boundary cells           ▷ suppress artefacts at domain edges
13:  Render: RENDERDENSITY;
14:  if showField then DRAWFIELD; DRAWAXES
15:  end if
16: end procedure

```

Further reading.

- J. Stam, *Stable Fluids*, SIGGRAPH 1999.
- E. Guendelman et al., *Semi-Lagrangian Controllable Fluids*, ACM TOG 2012.
- R. Bridson, *Fluid Simulation for Computer Graphics*, 3rd ed., CRC Press, 2015. A concise, code-oriented textbook that covers projection solvers, semi-Lagrangian advection, vorticity confinement, and GPU tips.
- J. Kim and P. Moin, “Application of a fractional-step method to incompressible Navier–Stokes equations,” *J. Comp. Phys.*, 59 (1985) 308–323. The classic finite-difference projection paper used by many CFD codes.